

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I C.ELIYAZAR/192311162 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

C. Eliyazar

Signature of the student:

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.

Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB).

Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Solution :

1. Problem Identification

The SHA implementation produces different hash values for the same input, indicating errors in the algorithm implementation.

Errors Identified

Error 1: Incorrect Message Padding

SHA algorithms require padding according to the standard.

Possible mistakes:

- Missing '1' bit after the message.
- Incorrect number of zero bits.
- Original message length not appended correctly.
- Padding not aligned to 512-bit blocks.

Impact

- Different padded messages produce different hashes.

Error 2: Incorrect Block Processing

SHA processes data in 512-bit blocks.

Possible issues:

- Wrong block size.
- Missing blocks during processing.
- Incorrect loop iteration.

Impact

- Final digest becomes incorrect.

Error 3: Bitwise Operation Errors

SHA heavily depends on:

- AND (&)

- OR (|)
- XOR (^)
- NOT (~)
- Left Rotate

Common mistakes:

- Using left shift instead of rotation.
- Missing 32-bit masking.
- Incorrect XOR logic.

Impact

Intermediate values become incorrect, producing invalid hashes.

Error 4: Endianness Problems

SHA uses Big-Endian byte ordering.

Common mistake:

Reading bytes using Little-Endian format.

Example

Correct:

01 02 03 04

Stored as

0x01020304

Incorrect

0x04030201

Impact

Hash value changes completely.

Root Cause Analysis

Problem	Root Cause	Effect
Wrong padding	Incorrect padding algorithm	Invalid hash
Missing blocks	Incorrect loop	Partial hashing
Bitwise errors	Wrong rotate/shift	Wrong digest
Endianness	Little-endian conversion	Different output

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.

Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption).

Analyze the performance vs security trade-offs in TLS optimization.

Solution :

2. Debugging Process (25%)

Step 1

Verify message length.

Original Length = $\text{len}(\text{message}) \times 8 \text{ bits}$

Step 2 : Append one binary 1

10000000

Step 3

Append zero bits

Until total length becomes

448 mod 512

Step 4 : Append 64-bit message length.

Step 5

Split into

512-bit blocks

Step 6

Process every block.

for each block:

 Expand schedule

 Perform 64 rounds

 Update hash values

Step 7

Verify output using Python hashlib.

import hashlib

print(hashlib.sha256(b"Hello").hexdigest())

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.

Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution.

Evaluate the impact of randomness quality on signature security.

Solution :

```
import hashlib

def sha256_hash(filename):
    sha = hashlib.sha256()
    with open(filename, "rb") as file:
        while True:
            data = file.read(8192)
            if not data:
                break
            sha.update(data)
    return sha.hexdigest()

file_name = "sample.txt"
print("SHA-256 Hash:")
print(sha256_hash(file_name))
```

Code Explanation

hashlib.sha256() -- Creates SHA-256 object.

read(8192) -- Reads file in 8 KB chunks.

sha.update() -- Processes every chunk.

hexdigest() -- Returns final hexadecimal hash.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic

system.

Solution :

Optimization Techniques (20%)

Optimization 1

Process data in chunks.

Old Method

Read entire file

Problem

High memory usage.

Improved Method

Read 8192 bytes at a time.

Benefit

Works efficiently even for files larger than **50 MB**.

Optimization 2

Avoid duplicate hashing.

Hash each block only once.

Optimization 3

Use optimized library.

hashlib

implemented in C.

Benefits

- Faster execution
- Lower CPU usage
- Better reliability

Optimization 4

Reuse buffers.

Avoid repeated memory allocation.

Performance Analysis

Method	Memory	Speed
Entire File	High	Moderate
Chunk Processing	Low	High
hashlib	Very Low	Very High

Time Complexity

Let

n = file size

Reading:- $O(n)$

Hash computation:- $O(n)$

Overall : - Time = $O(n)$

Memory :- $O(1)$

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification.

Optimize throughput for large file transfers (≥ 100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

Solution :

Security Analysis (20%)

SHA vs Basic Checksum

SHA-256	Basic Checksum
Cryptographically secure	Not secure
Collision resistant	Easy collisions
Detects tampering	Weak detection
One-way function	Easily manipulated
Used in digital signatures	Rarely used

Why SHA Improves Integrity

If one bit changes,

Original File

Hello

SHA-256

185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

Modified File

hello

SHA-256

2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

A single character change produces a completely different hash (the **avalanche effect**), making tampering easy to detect.

Advantages of the Optimized Solution

- Correct SHA padding implementation.
- Correct 512-bit block processing.
- Proper bitwise operations.
- Correct Big-Endian handling.
- Efficient processing for files larger than 50 MB.
- Low memory usage through chunk-based reading.
- Uses the optimized hashlib implementation.
- Produces consistent and accurate hash values.

Conclusion

The inconsistent SHA output was caused by incorrect **message padding, block processing, bitwise operations, and endianness handling**. After correcting these issues and optimizing the implementation using **chunk-based processing** and Python's **hashlib** library, the algorithm generates consistent SHA-256 hashes while efficiently processing large files. Compared to basic checksum methods, SHA-256 provides significantly stronger integrity protection due to its collision resistance, avalanche effect, and one-way cryptographic design, making it suitable for modern security applications.